# ORBITAL MECHANICS-FUNDAMENTAL THEORY WITH APPLICATIONS

Final Project
MATH499

This project is an overview along with some examples of how to apply fundamental formulas and techniques to build planetary models. From the orbit equation and using least squares to get a state vector (as done in practice), to applying some numerical techniques that provide insight on the behavior of the two and three-body problems.

Samuel Victor Flores

AMERICAN MILITARY UNIVERSITY

January 2025

# INTRODUCTION

This is a culmination of learning over the course of the Bachelor of Science in Applied Mathematics concentrating in Statistics. Several Examples of Basic Orbital Mechanics will be presented showcasing the applications of Geometry, Physics, Linear Algebra, and numerical modeling using Bezier curves and Least Squares.

Initial formulas will be introduced and developed into a straightforward approach to generating models of circular and elliptical orbits. Comparisons will be made between the two and three-body problems, where the goal is to regress a 3B problem into a 2B problem. Then, a guide on how to apply linear least squares to gather an estimate of tangential velocity to generate an initial state vector for a circular orbit. Finally, an overview of the application of Rodriguez Rotation to estimate the orbital plane using observed data of the moon's ecliptic coordinates.

## Initial Formulas

To model a simple orbit, begin by first computing the ellipse parameters using the desired periapsis, apoapsis, and radius of the central body.

This example will use an orbit with

$$P_o = 60, A_o = 800, R_e = 300$$

$$P_f = 800, A_f = 800$$

For demonstration purposes the value for GM = $3.98*10^8$ = u.

From here the eccentricities, momenta, and velocities can be computed.

$$e_n = \frac{(R_e + A_n) - (R_e + P_n)}{(R_e + A_n) + (R_e + P_n)} \; ; where\ n = \{o, f\}$$

The formula for angular momentum is

$$H_{\{P,A\}_n} = \sqrt{(R_e + \{P, A\}_n)(1 + e_n)u}$$

For the velocity at a certain instance

$$V_{\{P,A\}_n} = \frac{H_{\{P,A\}_n}}{R_e + \{P, A\}_n}$$

From here, we can get the famous 'DeltaV' by simply finding the difference between the final and initial velocities.

$DV_{f-o} = V_f - V_o$For example, to transfer from the initial orbit to the final orbit we would compute two DeltaV's. An example of such Burn Time calculations is at the end of this paper.

## Building the Model Ellipse

The necessary components of the ellipse include

The semi-major axis ‘a’

The semi-minor axis ‘b’

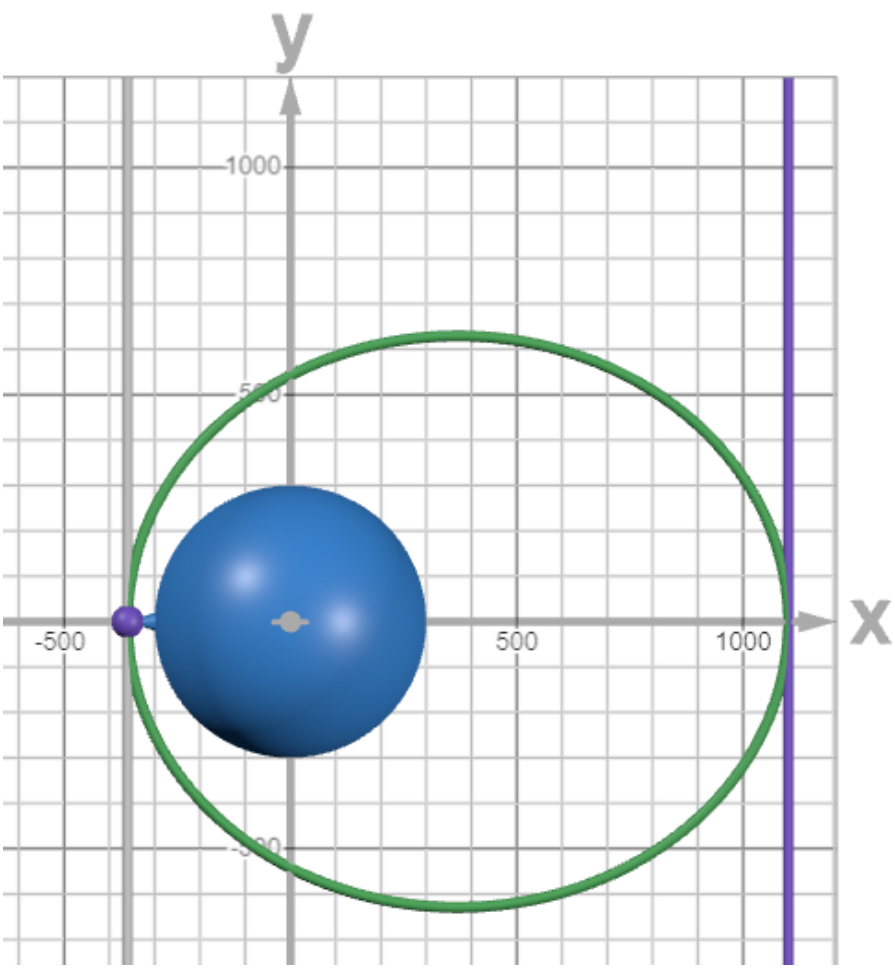
The distance from the center to a focal point ‘c’

To which,

$a = \frac{P+A+2R_e}{2}$ , half the diameter of the ellipse along the span of ‘a’( the ‘x’ axis).

$c = a - (P_0 + R)$ , half the diameter minus the distance from the center of the central body and the periapsis distance.

$b = \sqrt{a^2 - c^2}$, the semi-minor axis being derived from the Pythagorean theorem.




This ellipse traces the path of the transfer orbit ***before*** **circularization**.

However, when we track a point along this ellipse, the point will not follow Kepler’s laws, and the point will traverse the ellipse at a constant angular speed. Which is not characteristic of a non-circular orbit.

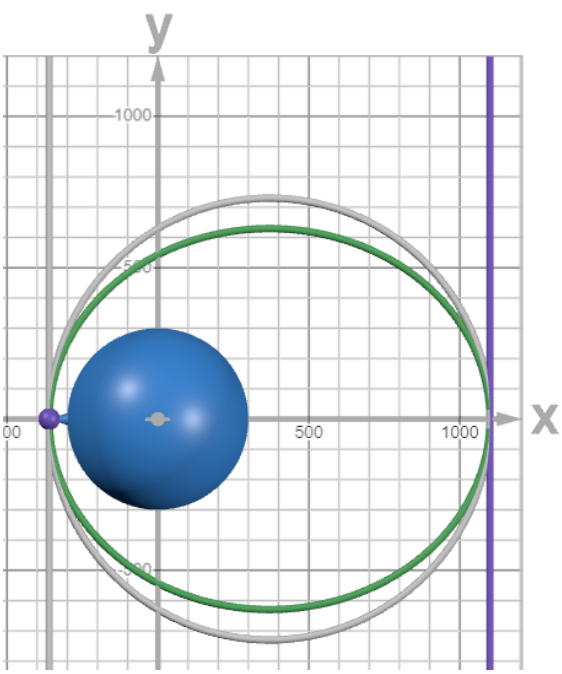
# Applying The Orbit Equation – 2 Body Problem

$$r = \frac{P_0 + R_e}{1 + ecos(wt)}\ , T_o = 2\pi\sqrt{\frac{a^3}{u}}\ , w = \frac{2\pi}{T_o}$$

*Orbit equation adopted from (Ling, n.d.)*

Above are the Orbit equation and the formulas for the period and mean angular speed of the orbit. 'w' is the mean angular speed, which traces a theoretical circular orbit about the edges of the ellipse.

The light grey circle(fig1) about the transfer orbit traces the mean orbit about the ellipse.



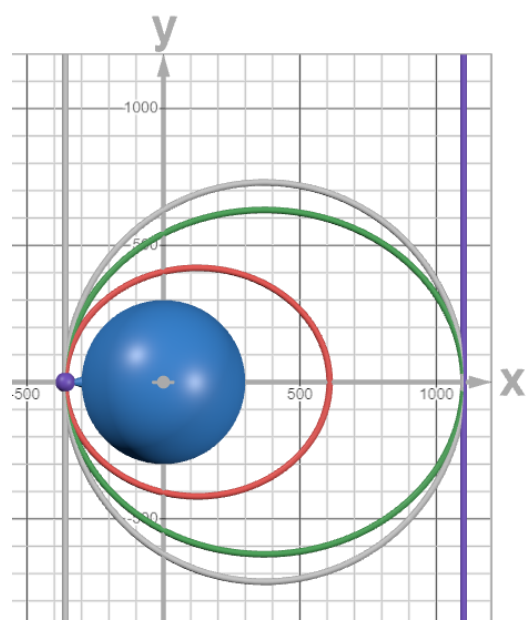

The form of this circle is,

$$(x - c)^2 + y^2 = a^2$$

Mapping 'r' to its components via

$$r \rightarrow \boldsymbol{r} = (rcos(wt) + c, r\sin(wt))$$

produces an ellipse that does not match the ellipse we intend to plot (fig2-red curve). To obtain the desired model, we can 'size up' this ellipse so we produce the desired mapping. To do this we form a set of vectors. The first traces the fundamental geometry of the ellipse, the second traces the ellipse, while informed by the orbit equation.




$$\boldsymbol{O} = \begin{cases} \text{a}\cos(wt) + c\ , b\sin(wt) \\ n_1 r_o \cos(wt) + d\ , n_2 r_o \sin(wt) \end{cases}$$

Where $d = A_o - P_o$

The 'n' constants are factors that when solved will 'size up' the ellipses together. By inspection it was found that $n_1$ and $n_2$ should be equal to produce the correct ellipse.

To solve the 'n' constants, set the components equal and compute the algebra.

$$n_1 = (a + c - d) * \frac{1 + e_o}{P_o + R_e}, n_2 = \frac{b}{r}$$

Notice that the cosine term is set to cos (0), $n_2$ is omitted as mentioned before. Hence,

*Figure 1*

$$\boldsymbol{O} \rightarrow \boldsymbol{R}_o = (\, n_1 r_o \cos(wt) + d\, , n_1 r_o \sin(wt)\,)$$

Applying the same theory for the final orbit yields,

$$\boldsymbol{F} = \begin{cases} \mathrm{a}_1 \cos(w_1 t) + c_1\, , b_1 \sin(w_1 t) \\ n_3 r_f \cos(w_1 t) + d_1\, , n_4 r_f \sin(w_1 t) \end{cases}$$

$$\boldsymbol{F} \rightarrow \boldsymbol{R_f} = (\, n_3 r_f \cos(w_1 t) + d_1\, , n_3 r_f \sin(w_1 t)\,)$$

$$n_3 = (a_1 + c_1 - d_1) * \frac{1 + e_f}{P_f + R_e},$$

$$r_f = \frac{P_f + R_e}{1 + e_f \cos\,(w_1 t)}\, , T_f = 2\pi \sqrt{\frac{a_1^3}{u}}\, , w_1 = \frac{2\pi}{T_f},$$

$$e_f = \frac{\left(R_e + A_f\right) - (R_e + P_f)}{\left(R_e + A_f\right) + (R_e + P_f)},$$

A peculiar issue regarding the orbit equation was the mismatch between the fundamental geometry and the ellipse informed by the orbit equation({**O**} ). One final assessment goes to whether the orbit equation with respect to the angular momentum can be expressed in a way that produces the correct ellipse.

The orbit equation of the form $r_h = \frac{H_{P_o}^2}{\mu} * \frac{1}{1+ecos(wt)}$, produces the correct ellipse. Finally, we can write the vector $\boldsymbol{R_h} = (\, r_h \cos(wt) + c\, , r_h \sin(wt)\,)$.

In this example $\frac{H_{P_o}^2}{\mu} = 360 = P_o + R_e$.

Leaving two forms of the orbital vectors

Geometric: $\boldsymbol{R}_o = (\, n_1 r_o \cos(wt) + d\, , n_1 r_o \sin(wt)\,)$

Physical: $\boldsymbol{R_{h_o}} = (\, r_h \cos(wt) + c\, , r_h \sin(wt)\,)$

It goes to show the physical form often is those most prudent.

# Restricted 3-Body Problem

This model of a central body and moon system is generated by first computing a 2-body system using the methods above. Typically, when such a model is made, both bodies usually orbit around a center of gravity. For brevity, this property is not featured in paper. Nonetheless, in both cases the complete motions of the two-body system are fully computed, and the third body is 'dropped into the orbital pool'.

Assumptions:

The initial 2-body system is a closed system and can be fully defined.

The third body induces negligible forces over the other bodies.

*For an 'N' body problem, there may be an ordinal nature to generating a model. Where models for the first two 'most significant' bodies are computed, then adding the rest in order of 'significance', where the significance is measured through mass and respective distances from the 'star'.*

Set-up and Initial Conditions

**Step1:** Compute the initial 2-body problem.

There is a central body with a mass/radius equal to our earth's, and a moon with 1.2%mass and 1/6$^{th}$ radius of the central body (the moon…)

As mentioned before this model will not include the barycenter of the initial two-body problem. The orbit will revolve about the center of the central body.

In the case that a second body is very small compared to a planet, like a spacecraft, there would hardly be a barycenter to consider.

Applying the method above yields a single dataset tracing the motion of the second body *(two datasets featuring a barycenter)*.

This data is marked (Ex1, Ey1), ellipse 'x' and 'y'.

**Step 2:** Applying the 3$^{rd}$ body

The 3$^{rd}$ body will react to the predefined paths of the 2-body problem. The two bodies act and the third body react.

The third body has mass, initial position, velocity, acceleration and thrust.

Using the following formulas.

$$mass(t) = m(t) = m(t-1) - \dot{m}t;\ \dot{m}\ is\ the\ mass\ flow\ rate$$

$$\boldsymbol{R} = (x\ , y, z)$$

$$\boldsymbol{a}(t) = \sum_{1}^{n-1} \frac{\mu_i}{r_i^3} \boldsymbol{R_i} + \frac{\boldsymbol{F_{thrust}}}{m(t)}$$

$$\boldsymbol{v}(t) = \boldsymbol{V_{n-1}} + \ \boldsymbol{a}(t) \cdot dt$$

Where $r_i = |\boldsymbol{R_i}|$, and $R(t) = \boldsymbol{R}(t-1) + \frac{1}{2}\boldsymbol{a}(t) \cdot dt^2 + \boldsymbol{v}(t) \cdot dt$

Polar Coordinates:

LAT, LON angles with respect to the VPython 3D axis with zero point at (LON,0, LAT =0, Z=1)

$$x = rcos(LON)\cos(LAT)$$

$$y = rsin(LON)\sin(LAT)$$

$$z = \cos(LON)$$

$\mathbf{F}_{\text{Thrust}}$ (=**F**) also has initial direction in terms of LAT and LON. To model a natural object (asteroid, satellite), the mass flow rate and thrust can be set to zero.

$$F_x = Fcos(LON)\cos(LAT)$$

$$F_y = Fsin(LON)\sin(LAT)$$

$$F_z = F\cos(LON)$$

To model a closed orbit, we can compute the angular momentum and tangential velocity then begin the model at the initial point.

To get a glimpse of the performance of this model we can input the initial conditions of a 3-body problem into the 3-body problem.

Take the computations of a 2-body problem and place the satellite into the 3-body system.

Example.

Earth moon system, with masses ME and 0.012ME, radii at zero and 60*$R_e$.

Generate a two-body solution for this system.

Next the initial conditions of the 3rd body are

M = 7280kg, LAT,LON = 0,0, $V_0$ = 7.729km/s, radius 300km, with a tangential direction on the velocity.

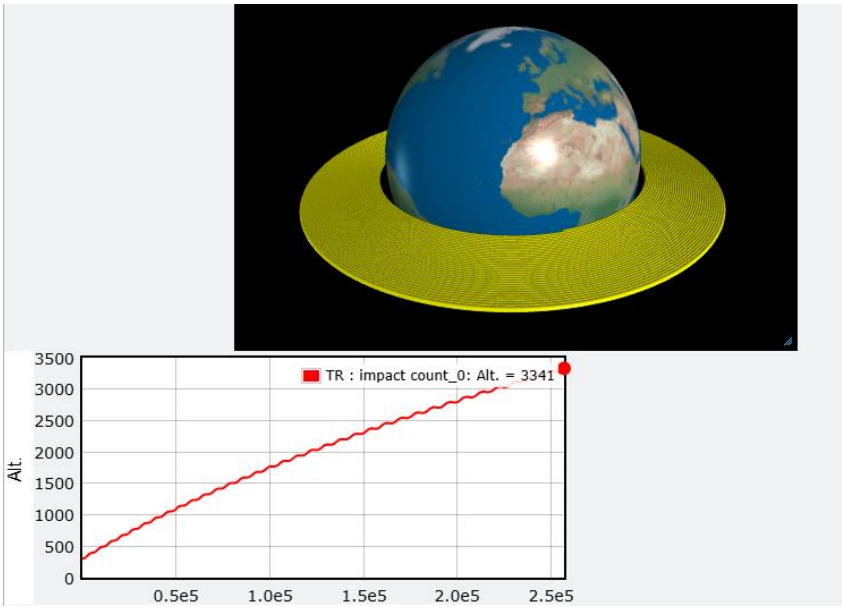



*Figure 2*

The initial conditions are computed using the 2-body formulas. We know the orbit is expected to be circular. In Fig3 the orbit grows as time moves forward but holds a somewhat circular form.

Other more elliptical orbits exhibit a motion where the periapsis and the apoapsis increase, a clear indication of the conservation of momentum.

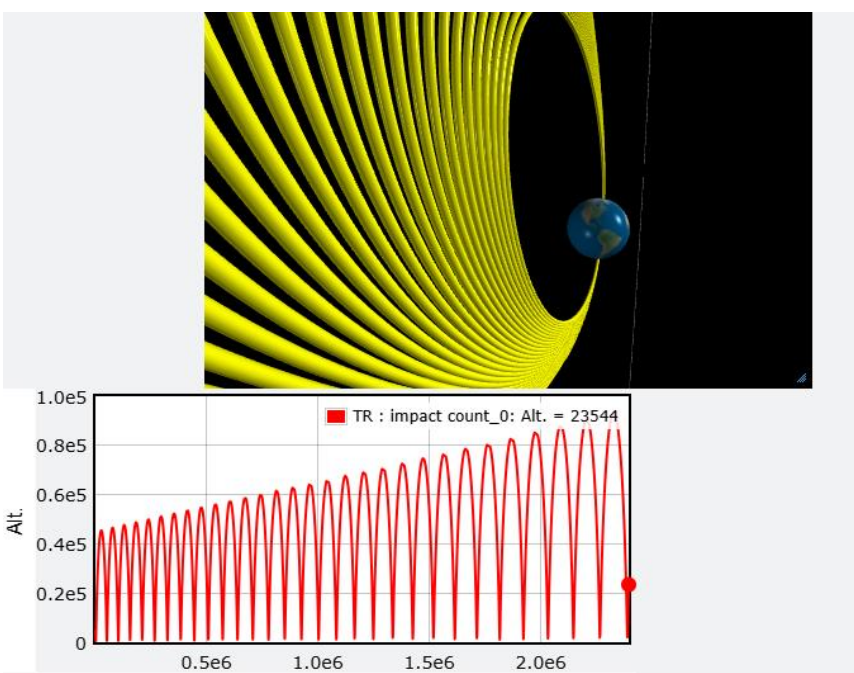



*Figure 3*

Fig4 shows a more elliptical orbit. The altitude profile behaves similarly with the circular orbit.

One way to verify the viability of this 3B-Problem. Is to apply the initial conditions of another two body problem. Fig3 is an example of such an application and it's visible that the behavior of the altitude is positive and non-linear. Therefore, corrective measures are in order.

# Linear and Bezier Curve Corrective Measures

Step1: Run the 3B-model completely.

Step2: fit a linear regression (good for very short time spans)

Step3: Fit a four-point Bezier Curve across the altitude profile.

The formula for a four-point Bezier curve is:

$$\left(B_x, B_y\right) = \Big(I\big(\mathrm{I}(\mathrm{I}x_0 + tx_1) + t(\mathrm{I}x_1 + tx_2)\big) + t\big(\mathrm{I}(\mathrm{I}x_1 + tx_2) + t(\mathrm{I}x_2 + tx_3)\big),$$
$$\mathrm{I}\big(\mathrm{I}(\mathrm{I}y_0 + ty_1) + t(\mathrm{I}y_1 + ty_2)\big) + t\big(\mathrm{I}(\mathrm{I}y_1 + ty_2) + t(\mathrm{I}y_2 + ty_3)\big)\Big)$$

(DESMOS.com, n.d.)

Where $0 \leq t \leq 1 \rightarrow I = 1 - t \rightarrow \ t = \frac{i}{s}$

'i' is the interactive index of the code, 's' is the timespan of the model. (Lawrence, 2018) (Weisel, 2010)

In this next example, we know the orbit is circular as computed by the 2B-Problem. Likewise, as seen in Fig3, the radius increases at a non-linear rate. For short term forecasts, we can simply take the LR of the profile and subtract from the model radius and re-compute the coordinates.

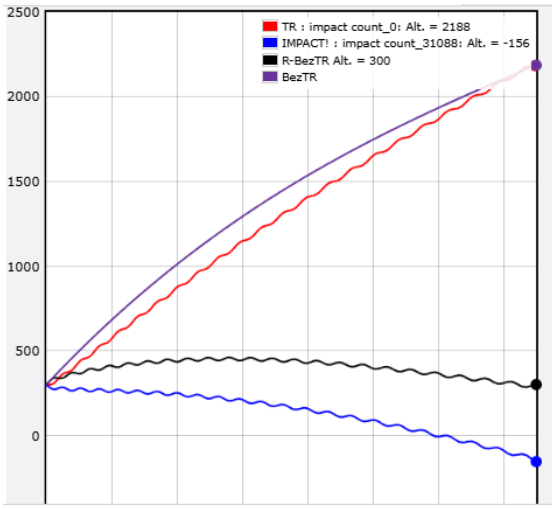
For longer term forecasts, a Bezier curve fit shows to stabilize very circular orbits.

The initial conditions for the 3rd body:

```
#initial position/motion of craft(3rd body)
bR0 = Re+300
LON = 60*pi/180
LAT = 0*pi/180# HI ADIZ @ LON -150, LAT -44

bR0x = bR0*sin(LON)*cos(LAT)
bR0y = bR0*sin(LON)*sin(LAT)
bR0z = bR0*cos(LON)
craft.pos = vector(bR0x,bR0y,bR0z)

Thrust = 0# kN
bRv0 = 7.72988176989
bRa0 =  0

AV_lat =  90*pi/180
AV_lon = 90*pi/180
```

*Figure 4 – bear in mind the code has to cater to the VPython 3d space.*

The resulting curves after running and fitting.




The model altitude(red line) behaves as mentioned. The blue line is the linear regression correction($R(t) - \beta_1 t$) and the black curve is the Bezier correction. For this very circular orbit the Bezier correction appears to stabilize the altitude and thus regresses the model towards the "2B solution". *(notice at the end of the cycle the final altitude is equal to the starting*)

**Improved Bezier Correction**

For highly elliptical orbits, however the initial Bezier approach could use some improvements.

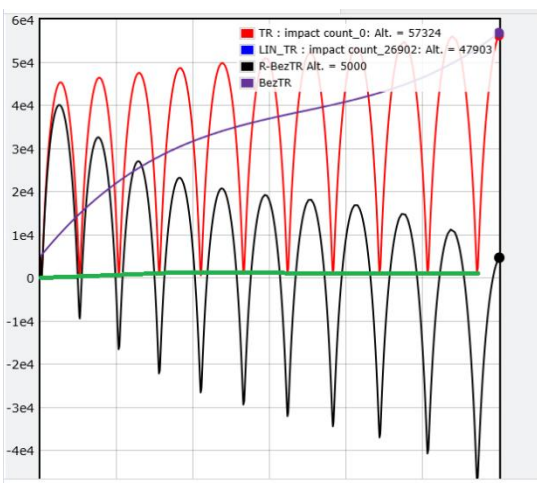



*Figure 5*

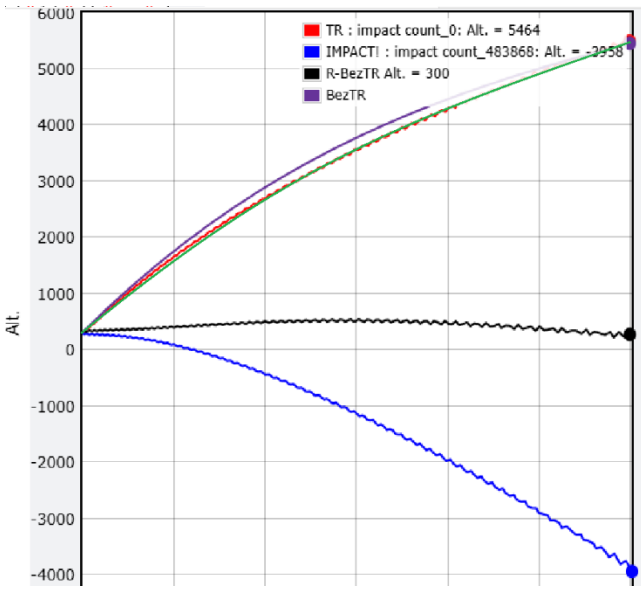
The purple line(Bezier Curve) draws straight through the chart. As it was meant to do… But we can see how this affects the Bezier Altitude (black line) in that over time the error gets worse. The line light green line (drawn by hand) is the next approach to refine the Bezier curve.

Step1: Run the model

Meanwhile recording the occurrence of each periapsis.

Step2: Fit the Bezier curve to the sequential periapsis points. This should minimize the error while stabilizing the elliptical orbit.

The logic for this approach is to correct the periapsis, the apoapsis will be corrected proportionately.




Likewise for the highly circular orbit, the Bezier Curve can be improved by drawing through the inflection points.

Fig7 is the same circular orbit but with a longer timespan for the model.

The light green line (hand drawn) shows what the Bezier curve through the inflection ought to look like. The code and theory for this process is intensive and out of the scope of this project.

*Figure 6*

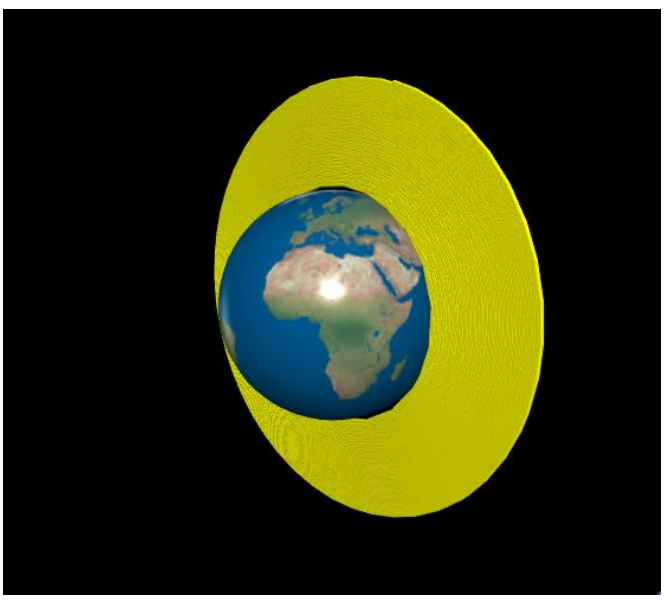


*Figure 7 - 3B model for the circular orbit*

**'Higher Order' Bezier Corrections-(Speculation)**

In short:

Compute the Bezier curve.

Then fit an appropriate Bezier curve into the first curve.

Iterate until the altitude profile becomes stationary.

Fit a non-linear least regression to model the trigonometric behavior(if it exists).

Then finally, subtract(diffuse) the 3B model to regress towards the 2B solution.

$$R = R_0 - \prod_{1}^{n} B_{n-1}\left[B_n[R_0]\right](1 + \beta_1 \cos(wt) + \beta_2 \sin(wt))$$

$\prod_1^n B_{n-1}\left[B_n[R_0]\right]$ : Represents the interactive quantity of the final Bezier curve that drives the model to stationarity.

$\beta_1 \cos(wt) + \beta_2 \sin(wt)$ : The negative of the least-squares non-linear regression, ruling out the least significant factor.

**Machine Learning Idealism – A speculative prospect to predicting errors**

The accumulation of errors in the 3B problem. It may be possible to develop a model that applies to thousands of theoretical/observed orbits, then applies a Neural Network type of model to predict the most likely error behavior.

## Statistical Orbit Determination

The proposed method to determine the orbit of a satellite involves the recording of observations of the object, either in very close time frames, or at the same time over a period of days. Ideally a combination of both sorts of observations is better.

Step1: Record Time(ZULU), Right Ascension and Declination

Step2: Convert to terrestrial coordinates (LAT/LON)

Step3: Compute the equation of the plane.

Step4: Compute the 6 main Orbital parameters

Step7: Build the ellipse as done in the 2B-Problem

Step8: Apply the Rodriguez rotation and rotation about the z-axis to tilt and rotate the orbit into place.

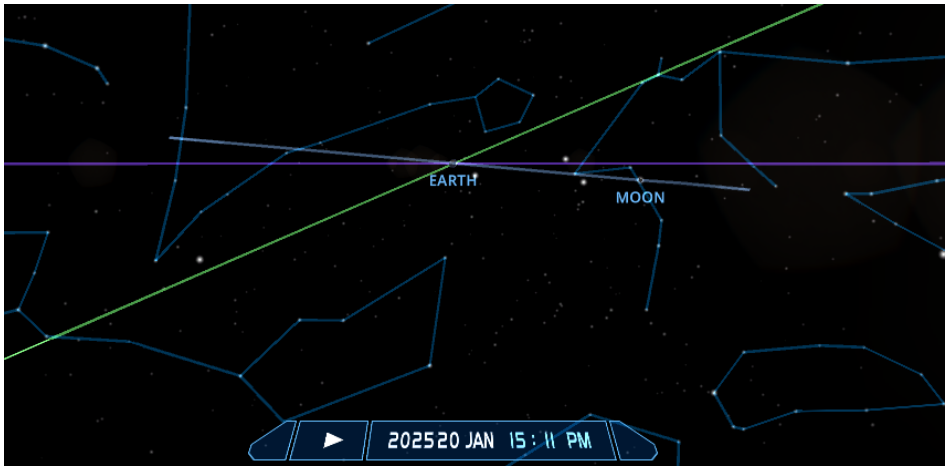



*Figure 8*

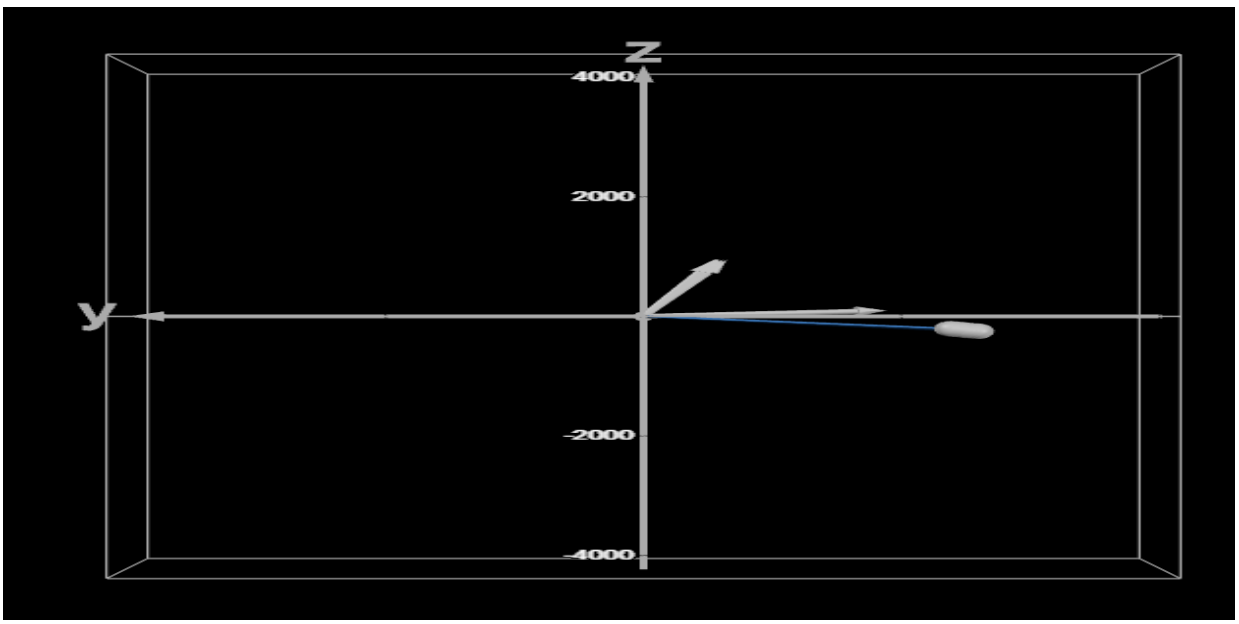



*Figure 9 – DESMOS - https://www.desmos.com/3d/51ax2m5soc*

Observations made with (https://theskylive.com/where-is-moon, n.d.) on January 20th, 2025, produced the vectors seen in Fig10. The observations were converted from Right Ascension and Declination to LAT and LON, then the 3D model was compared to simulator. (https://www.solarsystemscope.com/, n.d.)

In Fig 9 the purple line is the ecliptic plane(orbit about the sun), and the green line is the equatorial plane(about the rotation of the earth.)

To produce the similarity of these two models. The RH and DEC are converted to rectangular coordinates using

$$Z = Rsin(RH)\sin(DEC)$$

$$Y = Rsin(RH)\cos(DEC)$$

$$X = Rcos(RH)$$

Where 'X' points towards the first point of Aries, the point where the ecliptic plane intersect the equatorial plane located just under the constellation Pisces.

**Using Least squares to compute Tangential Velocity**

Step1: Compute angles between successive data points using

$$\Delta\vartheta = \arccos\left(\frac{A \cdot B}{|A||B|}\right), \omega_n = \frac{\Delta\vartheta}{\Delta t}$$

Step2: Run a Least squares LR to approximate the mean radius and mean angular speed.

Step3: Assuming strong significance is obtained for the intercept coefficient. Compute the Tangential velocity using $\widehat{\omega}\hat{r} = \hat{V}\perp$.

The state vector in terms of radius and Tangential speed is $\begin{bmatrix} \hat{r} \\ \hat{V}\perp \end{bmatrix}$.

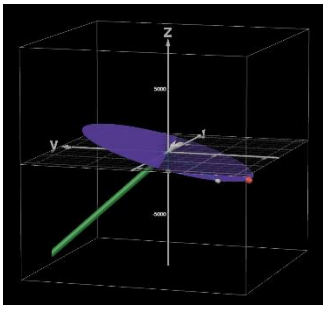
# Application of Rodriguez Rotations

The Rodriguez rotation matrix can be used to apply rotations on a set of points such that, in this case the orbit can exhibit an inclination and setting about the north pole, so the ascending and descending nodes are placed appropriately.

Step1: derive and model the initial orbit ellipse.

Step2: Apply the Rodriguez rotation about the north pole to place Ascending/descending nodes. This is based on the longitude of either apoapsis or periapsis.

Step3: Apply another R-Rotation about the axis that spans along the ascending and descending nodes. Computing the cross product between data points will produce an appropriate axis also( as seen in the presentation).




Another method is to compute the cross product between two measurements. In Fig11, two sets of measurements are made 3 days apart. It could be possible to interpolate the 'best' axis using the method of your choice. Here only the last and first measurements are used to produce the axis of rotation.

The formula for the Rodriguez Rotation Matrix is:

$\boldsymbol{R} = \boldsymbol{I} + \sin(\vartheta)\,\boldsymbol{A} + (1 + \cos(\vartheta))\boldsymbol{A}^{\mathbf{2}}$ (Mortenson, 2021)

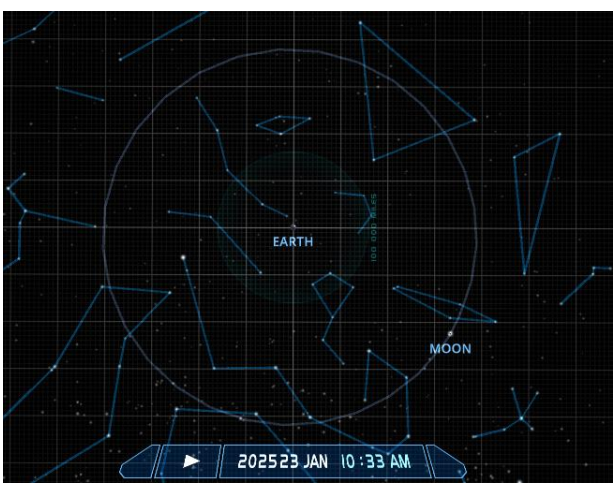
Where $\boldsymbol{A} = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix}$ (Mortenson, 2021)

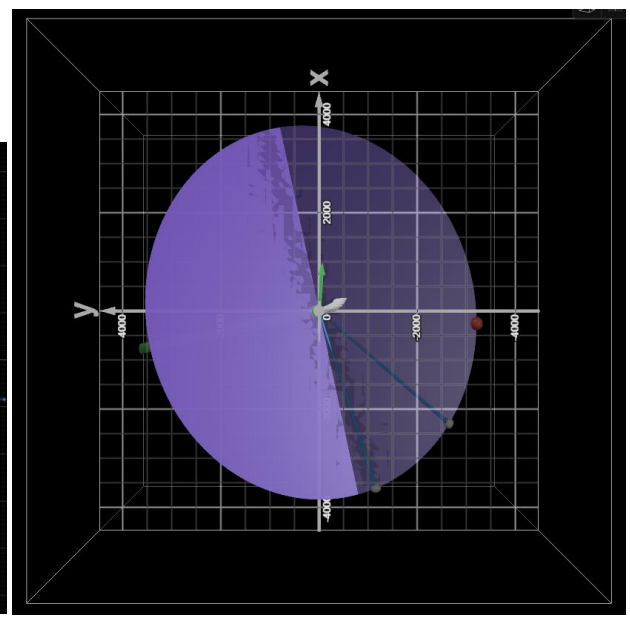
*Figure 10*

and $\boldsymbol{a} = \boldsymbol{P_1} \times \boldsymbol{P_2}$

$R_\vartheta = \boldsymbol{RP_t}$ gives the displacement about the axis of rotation.

Some extra photos to verify the RH accuracy. The inclination still appears too steep…



## Example Burn Time Calculations

Step1: copute intitial, transfer, then apply to burntime calculations

```
#periapse
P_o = 195.4#Kilometers
#apiapse
A_o = 264.9
#diamter of ellipse
D_o = P_o +A_o + 2*Re
# eccentricity
e0 = (Re + A_o - Re - P_o)/(2*Re +A_o +P_o)



## final orbit parameters
# input on dashboard
#periapse
P_f = 195.4
#apiapse
A_f = 300
#diamter of final ellipse
D_f = P_f +A_f + 2*Re
# eccentricity
ef = (Re + A_f - Re - P_f)/(2*Re +A_f +P_f)


# Momentum and Velocity/DV calculations Periapse
H_Po = sqrt((Re+P_o)*(1+e0)*mu)
H_Pf = sqrt((Re+P_f)*(1+ef)*mu) ## momentum at P-o for mid-transfer elliptic orbit
V_Po = H_Po/(Re+P_o)
V_Pf = H_Pf/(Re+P_f)### velocity at periapse_o! for mid-hohmman transfer.
DV_Pof = V_Pf - V_Po

# Momentum and Velocity/DV calculations Apiapse
H_Ao = sqrt((Re+A_o)*(1+e0)*mu)
H_Af = sqrt((Re+A_f)*(1+ef)*mu)
V_Ao = H_Ao/(Re+ A_o)
V_Af = H_Af/(Re+ A_f)
DV_Aof = V_Af - V_Ao


Thrust = 6197#N/s
M_dot = 2.1#kg/s
#initial Mass of spacecraft
M_o = 7259.72 #Kg

# from periapse to transition path
BT10 = M_o / ( Thrust/(DV_Pof*1000) +2.1  )
BT10


BT1 = M_o/( ( (Thrust-  (M_o-2.1*BT10)*(DV_Pof/BT10)) / (DV_Pof*1000) )  +2.1  )
BT1
```

Step2: Compute transfer and final orbit parameters then apply to final burn time calculations.

```
## Begin with circ. orbit parameters
#periapse
P_o = 195.4#Kilometers
#apiapse
A_o = 300
#diamter of ellipse
D_o = P_o +A_o + 2*Re
# eccentricity
e0 = (Re + A_o - Re - P_o)/(2*Re +A_o +P_o)



## final orbit parameters
# input on dashboard
#periapse
P_f = 300
#apiapse
A_f = 300
#diamter of final ellipse
D_f = P_f +A_f + 2*Re
# eccentricity
ef = (Re + A_f - Re - P_f)/(2*Re +A_f +P_f)


# Momentum and Velocity/DV calculations Periapse
H_Po = sqrt((Re+P_o)*(1+e0)*mu)
H_Pf = sqrt((Re+P_f)*(1+ef)*mu) ## momentum at P-o for mid-transfer elliptic orbit
V_Po = H_Po/(Re+P_o)
V_Pf = H_Pf/(Re+P_f)### velocity at periapse_o! for mid-hohmman transfer.
DV_Pof = V_Po - V_Pf## reversed for circuliarization

# Momentum and Velocity/DV calculations Apiapse
H_Ao = sqrt((Re+A_o)*(1+e0)*mu)
H_Af = sqrt((Re+A_f)*(1+ef)*mu)
V_Ao = H_Ao/(Re+ A_o)
V_Af = sqrt(mu/(Re+A_f))
DV_Aof = V_Ao - V_Af



M_1 = 7232.85#M_o - M_dot*BT1
# from apiapse to circularization
BT20 = M_1 / ( Thrust/((DV_Aof)*1000) +2.1  )
BT20

BT2 = M_1/ ( ( (Thrust-  ((M_1)-2.1*BT20)*(DV_Aof/BT20)) / (DV_Aof*1000) )  +2.1  )
BT2## correction
```

# Bibliography


*18- STATE VECTOR AND GEOCENTRIC EQUATORIAL FRAME*. (n.d.). Retrieved from https://www.spacerl.com/: https://www.spacerl.com/wp-content/uploads/2019/01/State-Vector.pdf

DESMOS.com. (n.d.). *Bezier Curves*. Retrieved from https://www.desmos.com/calculator/d1ofwre0fr

Fierz, W. (2018, October 2). *Application of Bézier Curves for Calculating Likelihood Ratios for Plasma Amyloid-β Biomarkers for Alzheimer's Disease*. Retrieved from https://pmc.ncbi.nlm.nih.gov/: https://pmc.ncbi.nlm.nih.gov/articles/PMC6176144/

Hali, F. (n.d.). *NUMERICALLY STABLE DIRECT LEAST SQUARES*. Retrieved from https://autotrace.sourceforge.net/: https://autotrace.sourceforge.net/WSCG98.pdf

*https://theskylive.com/where-is-moon*. (n.d.). Retrieved from https://theskylive.com/where-is-moon

https://www.localsiderealtime.com/. (n.d.). *What is Sidereal Time?* Retrieved from https://www.localsiderealtime.com/whatissiderealtime.html

*https://www.solarsystemscope.com/*. (n.d.). Retrieved from https://www.solarsystemscope.com/

Kubo, H. (2023, june 24). *Python Implementation of Ellipse Fitting*. Retrieved from https://medium.com/: https://medium.com/@hirok4/python-implementation-of-ellipse-fitting-e8801dbb16a8

Lawrence, J. (2018). *Celestial Coordinates*. Cambrdge, MA: The MIT Press.

Ling, S. J. (n.d.). *12.7 The Orbit Equation*. Retrieved from http://www.physicsbootcamp.org/: http://www.physicsbootcamp.org/The-Orbit-Equation.html

Mortenson, M. E. (2021). *Vectors and Matrices for geometric and 3D Modleing*. South Norwalk,CT: Industrial Press inc.

NASA. (n.d.). *Moon Fact Sheet*. Retrieved from https://nssdc.gsfc.nasa.gov/: https://nssdc.gsfc.nasa.gov/planetary/factsheet/moonfact.html

Rouch, T. (2023, june 13). *Least Squares Ellipse Fitting*. Retrieved from https://betterprogramming.pub/: https://betterprogramming.pub/least-squares-ellipse-fitting-4c039d7645

Saleh, A. H. (2020). *Investigation of the State Vectors and Prediction of the Orbital*. Retrieved from https://iopscience.iop.org/: https://iopscience.iop.org/article/10.1088/1742-6596/1664/1/012010/pdf

Strickland, J. (2022). *Orbital Mechanics using Python and R*. United States of America: United States Space Force.

Weisel, W. E. (2010). *Modern Orbit Dtermination*. Yalonda, CT: Aphelion.

Project Links:

- Rodriguez Model(Moon Tracker):
  - https://www.desmos.com/3d/kz9vhwhufa
- 2-Body Problem:
  - https://www.desmos.com/3d/89kaxktphk
- 3-Body Problem
  - Circular Orbit: https://trinket.io/library/trinkets/e69c22d984e3
  - Elliptical Orbit1(arbitrary start): https://trinket.io/library/trinkets/copy/2737512c87c2
  - Elliptical Orbit2: https://trinket.io/library/trinkets/2737512c87c2
  - Ballistic Missile SIM: https://trinket.io/library/trinkets/6f3134fd0ed7
- Least Squares Example: https://trinket.io/library/trinkets/2b0ffc52c045